

U-HAP: Unified Authorization over Heterogeneous Policies with Efficient Multi-Resource Verification in Kubernetes

Abstract—Kubernetes environments increasingly host multiple applications across pods and clusters, each governed by independently defined authorization policies such as RBAC, ABAC, and ACL. This heterogeneity subjects users to repeated and fragmented authorization checks when accessing multiple resources, resulting in increased latency, inconsistent decisions, and management complexity. While Single Sign-On (SSO) simplifies authentication, it does not address the challenge of unified authorization across heterogeneous policies. We propose U-HAP, a unified authorization framework for efficient and consistent multi-resource authorization across heterogeneous policies without requiring standardization. U-HAP introduces a universal graph-based representation that captures diverse policy models within unified semantics, reducing authorization to a reachability problem. By compiling resource-scoped policy artifacts and performing lightweight evaluation at request time, U-HAP eliminates redundant checks and ensures deterministic conflict resolution. We implement U-HAP as a Kubernetes webhook authorizer and evaluate it at scale. Experimental results demonstrate that U-HAP maintains near-constant authorization latency even as the number of namespaces scales, significantly outperforming traditional approaches. Furthermore, compilation-based optimization enables sub-linear scaling for complex policies, while decision caching provides additional speedups for repeated requests.

Index Terms—access control, Kubernetes, policy unification, directed acyclic graph, hash consing, RBAC, ABAC, ACL, webhook authorization, Zero Trust

I. INTRODUCTION

Kubernetes is the de facto platform for orchestrating containerized applications, including security-critical systems [1]. Yet authorization remains a major challenge: misconfigurations and excessive permissions frequently lead to vulnerabilities and full cluster compromise [2], [3].

Authorization in Kubernetes is inherently *multi-application, multi-resource*. A single workflow may touch multiple pods, namespaces, or clusters, each governed by independently defined policies (RBAC [4], ABAC, ACL [5], or custom) that operate under incompatible semantics. This results in repeated checks, inconsistent decisions, and operational inefficiency, even when SSO reduces authentication overhead.

Existing authorization mechanisms suffer from several fundamental limitations. First, semantic fragmentation arises because widely adopted policy frameworks such as RBAC, ABAC, and ACLs employ incompatible policy languages and evaluation semantics, leading to increased policy duplication and a higher risk of misconfiguration [6]. Second, conflicts among multiple authorization mechanisms are implicitly resolved based on evaluation order, resulting in non-

deterministic and potentially inconsistent access decisions. Third, runtime inefficiency persists due to reliance on repeated policy scanning, even when only a small subset of rules is relevant to a given request, thereby introducing unnecessary computational overhead. Although existing solutions attempt to alleviate some of these challenges, they remain insufficient in supporting efficient and scalable *multi-model authorization* across diverse systems and policies [7], [8].

We propose U-HAP, a *compilation-driven, index-based* authorization framework that addresses inefficiencies and inconsistencies in heterogeneous policy enforcement for cloud-native systems. The core idea is to shift computation from runtime to an offline compilation phase, where diverse policies are transformed into resource–action scoped indices. This enables an efficient *two-level pruning mechanism*: *policy-type pruning* activates only relevant policy classes (e.g., RBAC, ABAC, ACL, deny), while *token-driven pruning* further limits evaluation to rules matching user roles, groups, and attributes. Combined with a unified graph-based abstraction, U-HAP ensures consistent and deterministic authorization decisions while significantly reducing runtime overhead. Although implemented as a Kubernetes webhook authorizer, the design is general and applicable to other distributed environments.

This paper makes the following contributions. First, we formalize the problem of *multi-model, multi-resource authorization* and introduce a unified graph-based semantic model integrating RBAC, ABAC, ACL, and explicit deny rules within a single framework. Second, we develop a *compilation-driven architecture* that precomputes resource–action indices, eliminating costly runtime policy scanning and enabling efficient access to relevant artifacts. Third, we design a lightweight *two-level pruning strategy* that minimizes verification cost by restricting evaluation to structurally and contextually relevant policies. Finally, we implement and evaluate U-HAP as a Kubernetes webhook authorizer, demonstrating efficient, low-latency, and deterministic authorization with unified semantics and minimal overhead.

II. RELATED WORK

Security and access control in Kubernetes environments have been widely studied, particularly from the perspectives of system hardening and configuration correctness. Foundational guidelines such as the NSA/CISA Kubernetes Hardening Guidance [1] and subsequent compliance-oriented analyses [9]

emphasize secure control plane configuration, workload isolation, and least-privilege enforcement. However, empirical studies reveal that misconfigurations and excessive permissions remain pervasive in real-world deployments [2], [3], often enabling privilege escalation and even full cluster compromise. Recent tools such as EPScan [10] further confirm that excessive RBAC permissions in third-party applications continue to be a systematic security risk. Complementary work on formal verification [11] demonstrates that static policy analysis can prevent misconfigurations in Kubernetes clusters, yet such approaches address correctness of individual policies rather than unified authorization across mixed-model environments.

Kubernetes natively adopts Role-Based Access Control (RBAC) [4], which provides scalable permission management but lacks expressiveness for fine-grained, context-aware authorization. To address these limitations, policy-as-code frameworks enable declarative and dynamic policy enforcement, while Zero Trust architectures [12] promote continuous verification and context-aware access decisions in distributed cloud-native environments. Nevertheless, these approaches operate on independently evaluated policy models and require repeated evaluation across multiple enforcement layers, leading to redundant checks and increased latency in multi-resource authorization scenarios [6]. Recent optimization efforts such as PerfSPEC [13] improve enforcement efficiency through performance-aware profiling, and compile-time policy optimization techniques [14] reduce redundant rule evaluation. However, both approaches operate within individual policy models and do not address unified authorization across diverse policy types.

Beyond RBAC, Attribute-Based Access Control (ABAC) and standardized frameworks such as XACML [7] provide more expressive and fine-grained authorization capabilities. However, XACML relies on heavyweight policy decision point (PDP) architectures that are not well suited for lightweight, low-latency cloud-native systems. Prior work on compiling XACML policies into indexed evaluation structures [15] has demonstrated significant speedups over sequential evaluation, motivating the compile-time optimization approach adopted by U-HAP. Recent works explore graph-based representations of access control, modeling authorization as relationship inference over entities and attributes [8], [16]. Large-scale systems such as Zanzibar [17] demonstrate the scalability of relationship-based authorization, but require runtime graph traversal, introducing overhead proportional to graph complexity. Similarly, approaches that convert ABAC policies into RBAC models [18] improve interoperability but do not resolve the challenge of unified and efficient runtime authorization across diverse policy types.

Overall, existing works either focus on hardening and misconfiguration detection, enhancing policy expressiveness, or improving verification and enforcement mechanisms. However, they do not provide a unified authorization framework that efficiently supports diverse policy models (e.g., RBAC, ABAC, ACL, and deny rules) while minimizing redundant evaluation and ensuring scalable, low-latency multi-resource

authorization in cloud-native environments.

III. PROPOSED SYSTEM

A. System Model

U-HAP separates *policy compilation* (offline) from *request-time authorization* (online) to achieve low-latency, deterministic enforcement (Fig. 1).

System Entities.

- **User ($\mathcal{U}$):** Authenticated subject issuing requests with dynamic attributes (roles, groups, context).
- **Administrator:** Defines policies (RBAC, ABAC, ACL, Deny) via the U-HAP DSL.
- **Kubernetes API Server ($\mathcal{K}$):** Dispatches authorization queries via `SubjectAccessReview` (SAR).
- **U-HAP Webhook ($\mathcal{W}$):** Evaluates SAR requests using compiled artifacts and returns decisions.
- **DSL Loader ($\mathcal{L}$):** Parses and validates policies.
- **Policy Compiler ($\mathcal{B}$):** Compiles policies into resource-scoped artifacts $\{\mathcal{C}_{n,r,a}\}$.
- **Graph Registry ($\mathcal{M}$):** In-memory store for compiled artifacts, enabling $O(1)$ lookup.
- **Resources ($\mathcal{O}$):** Protected Kubernetes objects organized by namespace.

U-HAP decouples authentication from authorization by deferring user-specific bindings to request time rather than embedding them in precomputed structures. Identity, roles, and attributes are supplied via tokens at request time, enabling dynamic evaluation without recompilation.

Trust Assumptions. The API server and U-HAP components are trusted. Policies are assumed correct after validation. Adversaries may issue malicious requests but cannot tamper with compiled artifacts or the registry.

B. Formal Graph Model

U-HAP defines a unified graph abstraction for heterogeneous authorization policies. For each namespace–resource pair (n, r) , the system constructs

$$G_{n,r} = (V_{n,r}, E_{n,r}) \quad (1)$$

where $V_{n,r}$ and $E_{n,r}$ capture all policy semantics for resource r .

The vertex set $V_{n,r}$ comprises typed nodes: subject nodes $s \in \mathcal{S}$ (authenticated identities), role nodes $\rho \in \mathcal{R}$ (RBAC abstractions), gate nodes $g \in \mathcal{G}$ (ABAC predicates), deny nodes $d \in \mathcal{D}$ (explicit denials), and a distinguished resource node r . The edge set $E_{n,r}$ encodes relationships: (s, ρ) for subject–role assignments, (ρ_i, ρ_j) for role hierarchy, (ρ, r) for role-based permissions, (s, r) for direct ACL permissions, (s, g) and (g, r) for attribute-based constraints, and (s, d) , (d, r) for explicit deny semantics.

Well-formedness requires that (i) the role hierarchy is acyclic, (ii) all edges terminate at a gate, deny, or resource node, and (iii) deny nodes have highest precedence, ensuring deterministic conflict resolution.

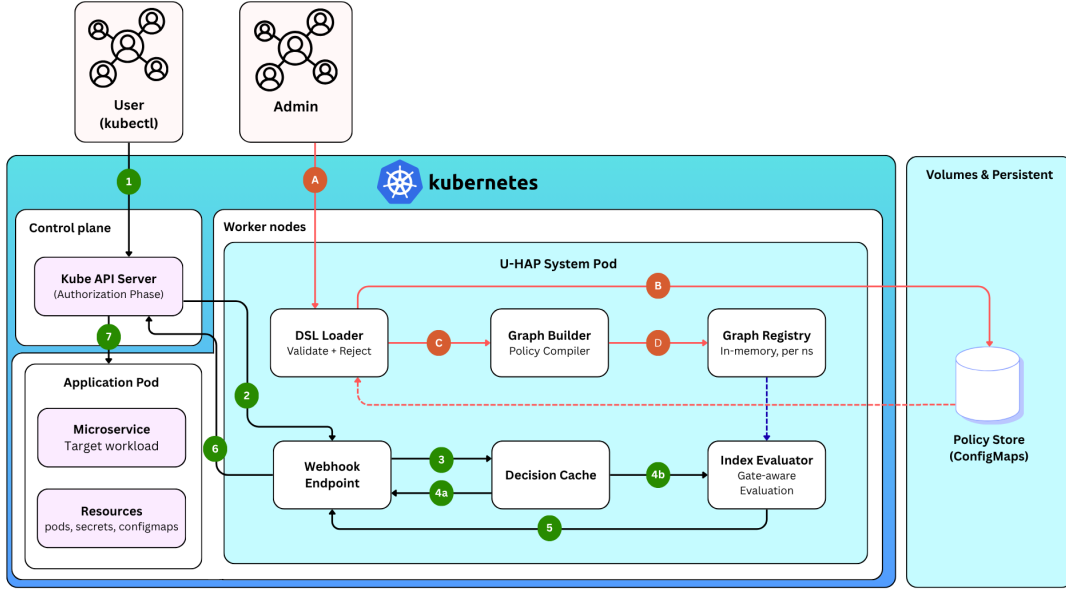


Fig. 1. U-HAP compilation-driven architecture on Kubernetes.

Authorization is defined as a constrained reachability problem over $G_{n,r}$. Given a request (s, a, r) with context $\mathbf{x}$, access is denied if a path $s \rightarrow d \rightarrow r$ exists; otherwise, access is granted if: (i) a direct edge (s, r) exists (ACL), (ii) a path $s \rightarrow \rho^* \rightarrow r$ exists through the role hierarchy (RBAC), or (iii) a path $s \rightarrow g \rightarrow r$ exists such that $g(\mathbf{x})$ is true (ABAC). If none apply, access is denied by default.

C. System Process

The proposed U-HAP system consists of three major phases as follows:

Phase 1: Setup (Step-Based Description): The setup phase initializes the U-HAP system and is executed once during deployment. It establishes the foundational structures required for policy compilation and runtime authorization.

Step 1: System Initialization.

Define the set of namespaces $\mathcal{N}$ and, for each $n \in \mathcal{N}$, the set of protected resources $\mathcal{O}_n$. Configure the Kubernetes API server to delegate authorization decisions via SubjectAccessReview (SAR) requests to the U-HAP Authorization Webhook. Normalize each incoming request into a unified representation:

$$\mathcal{Q} = \langle \tau, (n, r, a) \rangle \quad (2)$$

where τ is the user token and (n, r, a) denotes namespace, resource, and action.

Step 2: Component Deployment. U-HAP deploys four core components: the *DSL Loader*, which parses policy definitions; the *Policy Compiler/Graph Builder*, which transforms policies into graph-based structures; the *Graph Registry*, which stores compiled artifacts for constant-time lookup; and the *Authorization Webhook*, which handles runtime authorization requests.

The DSL Loader and Compiler operate offline during policy preparation, while the Registry and Webhook serve the runtime authorization path.

Step 3: State Initialization. The system initializes both a *persistent policy store* (e.g., Kubernetes ConfigMaps) to maintain policy definitions and an *in-memory registry* to enable efficient runtime access. For each (n, r, a) , a structured container of compiled artifacts is allocated as

$$\mathcal{C}_{n,r,a} = (G_{n,r}, I_{n,r,a}^{acl}, I_{n,r,a}^{rbac}, I_{n,r,a}^{abac}, I_{n,r,a}^{attr}, I_{n,r,a}^{deny}, \Psi_{n,r,a}), \quad (3)$$

where each component corresponds to a graph structure, policy index, or auxiliary metadata as defined in Table I.

Step 4: Token Trust Configuration.

Define the authenticated user token structure as:

$$\tau = \langle uid, \mathcal{R}_u, \mathcal{G}_u, \mathcal{A}_u, t_{exp}, \sigma \rangle \quad (4)$$

Where $uid, \mathcal{R}_u, \mathcal{G}_u, \mathcal{A}_u, t_{exp}, \sigma$ denote the authenticated user identity, the role set, the group set, the user attributes, the token expiration, and the cryptographic signature, respectively. Assume token authenticity is verified prior to authorization (e.g., via Kubernetes authentication layer). Define the supported policy types:

$$\Pi_n = \{\text{RBAC}, \text{ABAC}, \text{ACL}, \text{Deny}\}$$

with enforced schema and validation constraints.

Phase 2: Policy Compilation: The compilation phase transforms DSL policies into *resource-action-scoped runtime artifacts* consumed by the verifier in Phase 3. Rather than relying on runtime graph traversal, U-HAP compiles policies into verification-oriented indices enabling constant-time lookup, token-driven pruning, and short-circuit evaluation.

Input and Output. For $\Pi_n = \{\text{RBAC}, \text{ABAC}, \text{ACL}, \text{Deny}\}$, the compiler produces:

$$\mathcal{C}_n = \{\mathcal{C}_{n,r,a} \mid r \in \mathcal{O}_n, a \in \mathbb{A}\} \quad (5)$$

where each unit $\mathcal{C}_{n,r,a}$ aligns with the verification input $\langle \tau, (n, r, a) \rangle$, eliminating runtime filtering.

Compilation Procedure.

Step 1: Ingestion and IR Construction. Policies are parsed into:

$$\mathcal{IR} = (\mathcal{S}, \mathcal{R}, \mathcal{P}, \mathbb{A}, \mathcal{D}) \quad (6)$$

Where $\mathcal{S}, \mathcal{R}, \mathcal{P}, \mathbb{A}, \mathcal{D}$ denote the sets of subjects, roles, permissions, actions, and deny rules, respectively.

Step 2: Validation and Normalization. The DSL Loader enforces syntax, type safety, reference consistency, and acyclic role hierarchies. Role inheritance is reduced to transitive closure $\rho_i \Rightarrow^* \rho_j$.

Step 3: Resource-Action Decomposition. Policies are partitioned as $\Pi_n \rightarrow \{\Pi_{n,r,a}\}$, ensuring each artifact corresponds exactly to (n, r, a) .

Step 4: Artifact Generation. For each (n, r, a) , the artifacts shown in Table I are constructed.

TABLE I
COMPILED ARTIFACTS PER RESOURCE-ACTION SCOPE (n, r, a) .

Artifact	Definition	Notes
Graph	$G_{n,r}$	audit only
Deny	$I_{n,r,a}^{\text{deny}} = \{\text{scope}(d), \phi_d\}$	constant-time match
ACL	$I_{n,r,a}^{\text{acl}} = \mathcal{S}_{r,a}^{\text{acl}}$	membership check
RBAC	$I_{n,r,a}^{\text{rbac}} = \{\rho \mid \rho \Rightarrow^* (r, a)\}$	$\mathcal{R}_u \cap I_{n,r,a}^{\text{rbac}} \neq \emptyset$
ABAC	$I_{n,r,a}^{\text{abac}} = \{(g_\phi, \text{req}(\phi))\}$	$g_\phi : \mathcal{A}_u \rightarrow \{0, 1\}$
Attr	$I_{n,r,a}^{\text{attr}} : k \mapsto \{\phi \mid k \in \text{req}(\phi)\}$	selective evaluation
Summary	$\Psi_{n,r,a} = \langle \mathcal{S}_{r,a}^{\text{acl}}, \mathcal{R}_{r,a}^{\text{rbac}}, \Phi_{r,a}^{\text{abac}}, \mathcal{D}_{r,a} \rangle$	–

where $\text{scope}(d)$ is the set of principals to which deny rule d applies, ϕ_d is the condition predicate of deny rule d , and $\text{req}(\phi)$ is the set of attribute keys required by predicate ϕ . Each ABAC predicate ϕ is compiled into a DAG of GateNode objects via *hash consing*: structurally identical sub-expressions across policies share a single node, so any predicate appearing in k policies maps to exactly one GateNode in the shared DAG.

Step 5: Finalization and Registration. Redundant edges are removed, unreachable nodes are pruned, and consistency is verified. All $\mathcal{C}_{n,r,a}$ are then stored in the Graph Registry for $O(1)$ retrieval. ACL reduces to membership checking, RBAC to set intersection, ABAC to attribute-key-pruned predicate evaluation, and deny to prioritized filtering, eliminating global policy scans at runtime.

Phase 3: Request-Time Authorization: At runtime, U-HAP evaluates each request using a *policy-type-pruned, token-projected, index-optimized* pipeline over the compiled artifacts generated in Phase 2 and summarized in Table I. Given a request $\mathcal{Q} = \langle \tau, (n, r, a) \rangle$, the goal is to determine

whether the user represented by token τ is authorized to perform action a on resource r in namespace n . :contentReference[oaicite:0]index=0

Step 1: Request Reception and Token Extraction. The Authorization Webhook receives $\mathcal{Q} = \langle \tau, (n, r, a) \rangle$, where the validated token is

$$\tau = \langle uid, \mathcal{R}_u, \mathcal{G}_u, \mathcal{A}_u, t_{\text{exp}}, \sigma \rangle. \quad (7)$$

Here, uid denotes the authenticated user identity, $\mathcal{R}_u$ the role set, $\mathcal{G}_u$ the group set, and $\mathcal{A}_u$ the user and contextual attributes.

Step 2: Constant-Time Artifact Retrieval. Using the tuple (n, r, a) as a lookup key, the Webhook retrieves the compiled container $\mathcal{C}_{n,r,a}$ from the registry in $O(1)$ time. This container includes the graph, policy indices, and metadata defined in Table I. It also provides the fast-path activation descriptor

$$\Xi_{n,r,a} = \langle \lambda^{\text{deny}}, \lambda^{\text{acl}}, \lambda^{\text{rbac}}, \lambda^{\text{abac}} \rangle, \quad (8)$$

where each bit $\lambda^* \in \{0, 1\}$ indicates whether the corresponding policy type has at least one compiled rule for the requested (n, r, a) . Any class with $\lambda = 0$ is skipped entirely.

Step 3: Candidate Pruning. To avoid scanning irrelevant rules, U-HAP prunes the search space in two stages. First, *policy-type pruning* uses $\Xi_{n,r,a}$ to eliminate inactive policy classes. Second, *token-driven pruning* derives only the candidates relevant to the current user token:

$$\Gamma_{\text{deny}} = \{d \mid \text{scope}(d) \cap (\{uid\} \cup \mathcal{G}_u \cup \mathcal{R}_u) \neq \emptyset\}, \quad (9)$$

$$\Gamma_{\text{acl}} = (\{uid\} \cup \mathcal{G}_u) \cap I_{n,r,a}^{\text{acl}}, \quad (10)$$

$$\Gamma_{\text{rbac}} = \mathcal{R}_u \cap I_{n,r,a}^{\text{rbac}}, \quad (11)$$

$$\Gamma_{\text{abac}} = \bigcup_{k \in \text{dom}(\mathcal{A}_u)} I_{n,r,a}^{\text{attr}}(k). \quad (12)$$

Thus, Phase 3 evaluates only a compact request-specific subset rather than the full policy set.

Step 4: Decision Cache Check. Before evaluating rules, U-HAP checks a short-lived cache keyed by

$$\kappa = H(uid, n, r, a, \mathbf{b}_u, \mathbf{g}_u, \text{sig}(\mathcal{A}_u)), \quad (13)$$

where $\mathbf{b}_u$ and $\mathbf{g}_u$ are compact bit-vector encodings of user role and group memberships, $H(\cdot)$ is a collision-resistant hash function mapping the tuple to a κ , and $\text{sig}(\mathcal{A}_u)$ is a deterministic digest over the user attribute map $\mathcal{A}_u$. Thus, if κ is found in the cache, the system immediately returns the cached decision (e.g., allow) without further evaluation.

Step 5: Ordered Policy Verification. On a cache miss, U-HAP evaluates policies in fixed order

$$\text{Deny} \rightarrow \text{ACL} \rightarrow \text{RBAC} \rightarrow \text{ABAC} \rightarrow \text{Default Deny},$$

terminating upon a definitive decision.

(a) *Deny Check.* Deny rules take precedence. If any matches, the request is rejected:

$$\exists d \in \Gamma_{\text{deny}} : \text{match}_{\text{deny}}(d, \tau, n, r, a) = 1 \Rightarrow \text{DENY}, \quad (14)$$

where $\text{match}_{\text{deny}} = 1$ iff $\text{uid} \in \text{scope}(d)$ and $\phi_d(\tau, n, r, a) = 1$.

(b) *ACL Check*. If no deny applies, ACL authorization is checked via constant-time hash membership:

$$\text{uid} \in I_{n,r,a}^{\text{acl}} \vee \mathcal{G}_u \cap I_{n,r,a}^{\text{acl}} \neq \emptyset. \quad (15)$$

If true, the request is allowed.

(c) *RBAC Check*. RBAC is evaluated using precompiled bit-vectors for efficient role matching. Let $\mathbf{b}_u$ encode the roles in $\mathcal{R}_u$, and let $\mathbf{b}_{n,r,a}^{\text{rbac}}$ denote the authorized-role bit-vector for (n, r, a) after applying transitive closure over the role hierarchy. Authorization holds iff:

$$\mathbf{b}_u \wedge \mathbf{b}_{n,r,a}^{\text{rbac}} \neq \mathbf{0}. \quad (16)$$

This constant-time bitwise operation eliminates the need for runtime role-hierarchy traversal and ensures scalable authorization.

(d) *ABAC Check*. If RBAC fails, only relevant ABAC predicates are evaluated, sorted by cost:

$$\Gamma_{\text{abac}}^\uparrow = \text{sort}(\Gamma_{\text{abac}}, \text{cost}). \quad (17)$$

Authorization holds if any predicate succeeds:

$$\exists \phi \in \Gamma_{\text{abac}}^\uparrow : g_\phi(\mathcal{A}_u) = 1 \Rightarrow \text{ALLOW}. \quad (18)$$

Composite predicates are:

$$g_{\phi_1 \wedge \phi_2} = g_{\phi_1} \wedge g_{\phi_2}, \quad g_{\phi_1 \vee \phi_2} = g_{\phi_1} \vee g_{\phi_2}, \quad (19)$$

$$g_{\text{th}(k, \{\phi_i\})} = \mathbb{1} \left[\sum_i g_{\phi_i} \geq k \right]. \quad (20)$$

Step 6: Default and Cache Update.

$\text{Verify}(\tau, n, r, a)$ returns a cached decision on hit. Otherwise, it applies the above order: Deny $\rightarrow$ Allow (ACL/RBAC/ABAC) $\rightarrow$ Default Deny, and caches the result under κ .

IV. EXPERIMENTAL EVALUATION

This section evaluates the runtime authorization performance of U-HAP in a Kubernetes environment, where each *namespace* represents an isolated application or workload domain. We compare U-HAP with a conventional SSO-based approach, in which access rights are verified by evaluating application-specific policies stored in the cluster. To ensure a fair comparison, we measure only the time for policy lookup and verification against in-memory structures, excluding network and authentication overhead.

Experiment Setting. All experiments are conducted on the implemented U-HAP system (Python 3.11) on a machine equipped with an AMD Ryzen 9 7945HX CPU (16 cores, 32 threads), 32 GB DDR5-4800 RAM, running CachyOS Linux. Evaluation is single-threaded per request, reflecting the Kubernetes API server’s serialized

SubjectAccessReview (SAR) dispatch. Each data point is reported as the median of 1,000 timed iterations after 50 warm-up iterations.

Experiment 1 (Policy Verification Efficiency). We evaluate policy verification efficiency by varying the number of applications (i.e., namespaces representing independent policy sets) and comparing three configurations: U-HAP, U-HAP with caching, and a baseline SSO-based approach (policy checking time only). N is swept across $\{10, 50, 100, 200, 300, 500, 750, 1000\}$. In this experiment, each namespace contains 46 heterogeneous authorization rules: 10 RBAC, 20 ABAC, 10 ACL, 1 deny, and 5 role-hierarchy edges. ABAC predicates are defined using compound expressions with 10 logical gates (e.g., AND, OR, ATLEAST) and 11 attributes per rule, selected from a pool of 20 attribute keys representing organizational, location, security, and temporal contexts. Approximately 50% of ABAC atoms are shared across rules within each namespace.

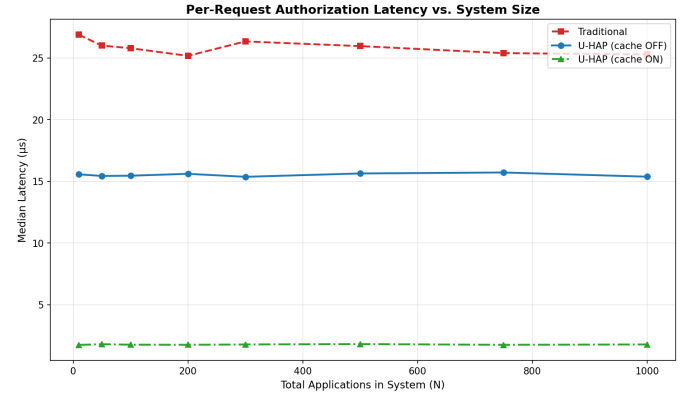


Fig. 2. Per-request authorization latency vs. total number of application namespaces.

Fig. 2 shows that all configurations maintain nearly constant latency as N increases from 10 to 1,000, indicating that adding applications does not impact the verification cost of a target application. Under average-case rule ordering, U-HAP achieves approximately $1.7\times$ lower latency than the baseline due to compiled evaluation: ABAC predicates are optimized via shared DAG evaluation, ACL checks use hash-based lookup, and RBAC is resolved through constant-time bit-vector matching. In contrast, the baseline must perform sequential policy scanning, including repeated role resolution and hierarchy traversal for RBAC. With caching enabled, U-HAP achieves an additional $\sim 14\times$ speedup by directly returning decisions for repeated requests, effectively bypassing policy evaluation. Overall, the results demonstrate that compilation and caching jointly provide complementary and significant performance improvements.

Experiment 2 (Policy Size Impact). We evaluate how different compiled representations affect authorization latency as the number of rules k increases. For each policy model, the number of rules k is varied from 5 to 320. In each configuration, only rules of the target type are present in the namespace.

ABAC rules use 10 gates per predicate with 50% cross-rule atom sharing. The test request uses average-case rule ordering: the matching rule is placed at the middle position ($\lfloor k/2 \rfloor$), so the traditional evaluator scans approximately $k/2$ rules on average. The decision cache is disabled to measure raw evaluation cost.

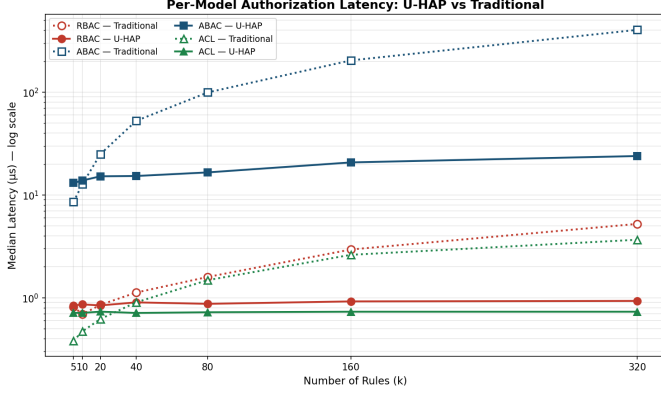


Fig. 3. Per-model authorization latency (log scale) vs. number of rules ($k=5-320$).

As shown in Fig. 3, the benefits of compilation increase with k across all models. ABAC achieves the largest gains: while the traditional evaluator grows rapidly due to repeated predicate evaluation, U-HAP scales more slowly via hash-consed DAG evaluation, eliminating redundant computations. This yields up to $16.9\times$ speedup at $k=320$, with a crossover around $k\approx 20$; at small scale ($k=5$), minor overhead ($0.7\times$) is observed.

For ACL and RBAC, U-HAP maintains near-constant latency using hash-set membership and bit-vector matching, respectively, while traditional evaluation grows linearly with k . This results in steady improvements, reaching $5.0\times$ (ACL) and $5.6\times$ (RBAC) at $k=320$, with crossovers at $k\approx 40$ and $k\approx 20$. Overall, performance gains become significant beyond the crossover point and increase with scale.

V. CONCLUSION

This paper presented U-HAP, a compilation-driven authorization framework that unifies RBAC, ABAC, ACL, and explicit deny into a single, consistent mechanism for Kubernetes. By shifting policy reasoning to an offline compilation phase, U-HAP eliminates redundant runtime scanning and enables efficient authorization through indexed artifacts and lightweight verification. Experimental results show that U-HAP achieves stable, low latency as the number of namespaces scales, outperforming conventional SSO-based approaches that rely on repeated policy evaluation. Compilation and decision caching provide complementary gains, achieving approximately $1.7\times$ speedup from compiled evaluation and up to $14\times$ for repeated requests. Per-model analysis further demonstrates improved efficiency across all policy types, with up to $16.9\times$ speedup for ABAC, $5.0\times$ for ACL, and $5.6\times$ for RBAC at $k=320$ rules. Future work includes extending U-HAP to

support cluster-wide and cross-namespace policies, enabling dynamic updates without full recompilation, and integrating with production-grade authorization engines for large-scale multi-cluster evaluation.

REFERENCES

- [1] NSA and CISA, “Kubernetes Hardening Guidance,” NSA Cybersec. Tech. Report, ver. 1.2, Aug. 2022. [Online]. Available: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF
- [2] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, “Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study,” *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, 2023, doi: 10.1145/3579639.
- [3] N. Yang, W. Shen, J. Li, X. Liu, X. Guo, and J. Ma, “Take Over the Whole Cluster: Attacking Kubernetes via Excessive Permissions of Third-Party Applications,” in *Proc. ACM CCS*, 2023, pp. 3048–3062.
- [4] G. Rostami, “Role-Based Access Control (RBAC) Authorization in Kubernetes,” *J. ICT Standardization*, vol. 11, no. 3, pp. 237–260, 2023, doi: 10.13052/jicts2245-800X.1132.
- [5] N. Farhadighalati, L. A. Estrada-Jimenez, S. Nikghadam-Hojjati, and J. Barata, “A Systematic Review of Access Control Models: Background, Existing Research, and Challenges,” *IEEE Access*, vol. 13, pp. 17777–17806, 2025, doi: 10.1109/ACCESS.2025.3533145.
- [6] M. S. Rahaman, S. N. Tisha, E. Song, and T. Cerny, “Access Control Design Practice and Solutions in Cloud-Native Architecture: A Systematic Mapping Study,” *Sensors*, vol. 23, no. 7, art. no. 3413, 2023.
- [7] OASIS, “eXtensible Access Control Markup Language (XACML) Version 3.0 Plus Errata 01,” OASIS Standard, 2017.
- [8] M. Yang, V. Atluri, S. Sural, and J. Vaidya, “A Graph-Based Framework for ABAC Policy Enforcement and Analysis,” in *Proc. DBSec*, 2024, pp. 3–23.
- [9] Z. Morić, V. Dakić, and T. Čavala, “Security Hardening and Compliance Assessment of Kubernetes Control Plane and Workloads,” *J. Cybersec. Privacy*, vol. 5, no. 2, art. no. 30, 2025.
- [10] Y. Gu, X. Tan, Y. Zhang, S. Gao, and M. Yang, “EPScan: Automated Detection of Excessive RBAC Permissions in Kubernetes Applications,” in *Proc. 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3199–3217.
- [11] A. Sissodiyi, E. Chiquito, U. Bodin, and J. Kristiansson, “Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters,” *IEEE Access*, vol. 13, pp. 141798–141813, 2025, doi: 10.1109/ACCESS.2025.3597504.
- [12] R. Chandramouli and Z. Butcher, “A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments,” NIST SP 800-207A, 2023.
- [13] H. Nguyen *et al.*, “PerfSPEC: Performance Profiling-Based Proactive Security Policy Enforcement for Containers,” *IEEE Computer*, 2025.
- [14] S. Kern, T. Baumer, S. Groll, L. Fuchs, and G. Pernul, “Optimization of Access Control Policies,” *J. Inf. Secur. Appl.*, vol. 70, art. no. 103301, 2022.
- [15] A. X. Liu, F. Chen, J. Hwang, and T. Xie, “Designing Fast and Scalable XACML Policy Evaluation Engines,” *IEEE Trans. Computers*, vol. 60, no. 12, pp. 1802–1817, Dec. 2011.
- [16] L. Ma *et al.*, “Research on Authorization Model of Attribute Access Control Based on Knowledge Graph,” in *Proc. UbiSec*, Springer, 2024, pp. 350–364.
- [17] R. Pang, R. Caceres, M. Burrows, *et al.*, “Zanzibar: Google’s Consistent, Global Authorization System,” in *Proc. USENIX ATC*, 2019, pp. 33–46.
- [18] M. Davari and M. Zulkernine, “Automatic Conversion of ABAC Policies for RBAC Systems,” in *Proc. 2023 IEEE Conference on Dependable and Secure Computing (DSC)*, 2023, pp. 1–7, doi: 10.1109/DSC61021.2023.10354157.